

EREDITARIETÀ IN LINGUAGGIO JAVA: SECONDA PARTE

Riprendiamo il concetto di ereditarietà...

Esempio 1

```
public class Car{
    boolean isOn;
    String licensePlate;
    ...
    public void turnOn(){...}
    public void turnOff(){...}
}

public class SDCar extends Car{
    boolean isSelfDriving; //attributo specifico della classe derivata
    ...
    void turnSDOn(){...} //metodo specifici della classe derivata
    void turnSDOff(){...} //metodo specifici della classe derivata
}
```

Nella classe principale Main (istanzio un oggetto di tipo SDCar):

```
SDCar c = new SDCar();
c.turnOn(); //invoco un metodo ereditato dalla classe Car
c.turnSDOn(); //invoco un metodo specifico della classe SDCar
```

Esempio 2:

```
public class Car{
    boolean isOn;
    String licensePlate;
    ...
    void turnOn(){...}
    void turnOff(){...}
}
```

```
public class SDCar extends Car{
    boolean selfDriving;
    void turnSDOn(){...}
    void turnSDOff(){...}
    void turnOn() -> OVERRIDE
    {
        turnSDOff();
    }
    void turnOff() -> OVERRIDE
    {
        turnSDOff();
    }
}
```

Viene ridefinito il corpo del metodo turnOn() e turnOff(), ovvero viene riscritto il corpo dei due metodi: OVERRIDE -> si ridefinisce un metodo della classe base.

Si vuole ora richiamare un metodo della classe base all'interno della classe derivata.

```
public class SDCar extends Car{
    boolean selfDriving;
    void turnSDOn(){...}
    void turnSDOff(){...}
    void turnOn()
    {
        turnSDOff();
        super.turnOn();
    }
    void turnOff()
    {
        turnSDOff();
        super.turnOff();
    }
}
```

super è una parola-chiave che indica un riferimento attraverso il quale la classe figlia (sottoclasse) si riferisce alla classe base.

super è, pertanto, una modalità con la quale una classe figlia identifica la sua classe genitore/base.

Es. `super.turnOn()` -> richiama il metodo “originale” definito nella classe base.

EREDITARIETA', INCAPSULAMENTO E DESCRITTORI DI VISIBILITA'

1. ATTRIBUTI PRIVATI (private) si usano quando:

- si desidera l'incapsulamento totale: gli attributi privati garantiscono che nessun'altra classe possa accedere o modificare direttamente il valore degli attributi, proteggendo i dati.
- I metodi setter e getter consentono l'accesso agli attributi privati da parte di altre classi, comprese ovviamente le classi derivate.

ACCESSO INDIRETTO AGLI ATTRIBUTI DA PARTE DELLE CLASSI DERIVATE (IN CASO DI EREDITARIETA') MEDIANTE I METODI SETTER E GETTER.

Le classi derivate interagiscono con tali attributi in modo controllato-

Esempio:

```
public classe Base{
    private int valore;
    //nessun getter e nessun setter
...
}
public class Derivata extends Base{
    public void modificaValoreAttributo()
    {
        valore = 10; //genera un errore di compilazione
    }
}
```

BISOGNA INSERIRE I METODI SETTER E GETTER NELLA CLASSE Base.

2. **ATTRIBUTI PROTETTI** (protected) si usano quando:

- si vuole implementare l'Ereditarietà in modo tale che le classi figlie devono accedere direttamente agli attributi della classe base.
L'utilizzo di protected consente alle classi figlie di accedere e modificare direttamente gli attributi della classe base.

Quando gli attributi sono dichiarati come protetti, l'accesso a tali attributi è consentito, oltre alle classi derivate, anche ad altre classi appartenenti allo stesso package.

Esempio:

```
public classe Base{  
    protected int valore;  
    //nessun getter e nessun setter  
...  
}  
  
public class Derivata extends Base{  
    public void modificaValoreAttributo()  
    {  
        valore = 10; //accesso diretto all'attributo  
    }  
}
```

3. **ATTRIBUTI PUBBLICI** (public) si usano quando:

- non si vuole garantire l'incapsulamento. Gli attributi definiti come public sono accessibili da qualsiasi altra classe, incluse le classi derivate.

Esempio:

```
public classe Base{  
    public int valore;  
    //nessun getter e nessun setter  
...  
}
```

```
public class Derivata extends Base{
    public void modificaValoreAttributo()
    {
        valore = 10;
    }
}
/*
accesso diretto all'attributo. Non vi è la necessità di
inserire nella classe Base i metodi setter e getter.
*/
```

ATTRIBUTI DICHIARATI DI TIPO PRIVATE

ESEMPIO DI IMPLEMENTAZIONE DELL'EREDITARIETÀ: COSTRUTTORE PRAMETRIZZATO SIA NELLA CLASSE BASE SIA NELLA CLASSE DERIVATA

```
public class Base{
    private int attributoBase;
    public Base(int attributoBase){
        this.attributoBase=attributoBase;
    }
    public int getAttributoBase(){
        return attributoBase;
    }
    public void setAttributoBase(int attributoBase){
        this.attributoBase= attributoBase;
    }
}
```

```
public class Derivata extends Base
{
    private String attributoDerivata;
    //costruttore classe figlia
    public Derivata(int attributoBase, String attributoDerivata){
        super(attributoBase); //super ha la funzione di chiamare il
                               costruttore della classe Base.
    }
}
```

/*

Perché passo come parametro al costruttore anche **attributoBase**?

Quando si crea un'istanza di Derivata, bisogna fornire i valori necessari per inizializzare sia gli attributi della classe base sia quelli della classe derivata. Ecco perché il costruttore di Derivata accetta **attributoBase** come parametro e lo passa al costruttore della classe base utilizzando `super(attributoBase)`.

La parola-chiave `super` si riferisce ad attributi della classe base nel senso che invoca il costruttore della classe base passandogli i parametri necessari. Il costruttore della classe base utilizza i valori di tali parametri per inizializzare gli attributi della classe Base.

*/

```
public String getAttributoDerivata()
{
    return attributoDerivata;
}

public void setAttributoDerivata(String attributoDerivata)
{
    this.attributoDerivata = attributoDerivata;
}
```

```
public void mostraAttributi()
{
    int attributoBase=super.getAttributoBase();
    System.out.println("attributo Base:" +attributoBase);
    System.out.println("attributo Derivata:" + attributoDerivata);
}

public void aggiornaAttributi(int nuovoV, String nuovaS)
{
    super.setAttributoBase(nuovoV);
    this.attributoDerivata=nuovaS;
}
}

public class Main{
    Public static void main(String[] args){
        Base b = new Base(50);
        Derivata d = new Derivata(42, "iniziale");
        d.mostraAttributi();
        //aggiorno attributi
        d.aggiornaAttributi(100, "finale");
        d.mostraAttributi();
    }
}
```

COSTRUTTORE NON PARAMETRIZZATO NELLA CLASSE BASE E COSTRUTTORE NON PARAMETRIZZATO NELLA CLASSE DERIVATA

```
public class Base
{
    private int eta;
    private String nome;
    public Base()
    {
        this.eta=0;
        this.nome="nome predefinito";
    }
    public int getEta()
    {
        return eta;
    }

    public void setEta(int eta)
    {
        this.eta=eta;
    }
    public String getNome(){
        return nome;
    }
    public void setNome(String nome)
    {
        this.nome=nome;
    }
}
```


/* NOTA BENE: poiché gli attributi vengono inizializzati direttamente nel costruttore, è possibile non indicare i metodi setter per gli attributi.

*/

```
public class Derivata extends Base
{
    private String corso;
    public Derivata()
    {
        super(); //chiamata del costruttore non parametrizzato della classe Base
        this.corso= "corso base";
    }
    public String getCorso()
    {
        return corso;
    }

    public void setCorso(String corso)
    {
        this.corso=corso;
    }
}
```

```
public class Main{
    Public static void main(String[] args){
        Derivata d = new Derivata();
        System.out.println("Nome:" + d.getNome());
        System.out.println("Corso:" + d.getCorso());
    }
}
```

**COSTRUTTORE NON PARAMETRIZZATO NELLA CLASSE BASE E
COSTRUTTORE PARAMETRIZZATO NELLA CLASSE DERIVATA**

```
public class Base{
    private String nome;
    public Base(){
        this.nome="nome predefinito";
    }
    public String getNome(){
        Return nome;
    }
    public void setNome(String nome){
        this.nome=nome;
    }
}

public class Derivata extends Base{
    private String corso;
    //Costruttore parametrizzato
    public Derivata(String corso)
    {
        super();
        this.corso=corso;
    }
    public String getCorso()
    {
        return corso;
    }
    public void setCorso(String corso)
    {
        this.corso=corso;
    }
}
```

```
public class Main{  
    Public static void main(String[] args){  
        Derivata d = new Derivata("Informatica");  
        System.out.println("Nome:" + d.getNome());  
        System.out.println("Corso:" + d.getCorso());  
    }  
}
```

ESEMPIO DI CODICE IN CUI NELLA CLASSE DERIVATA VIENE DEFINITO UN COSTRUTTORE DI COPIA

```
public class Veicolo  
{  
    private String marca;  
    private int anno;  
    // Costruttore  
    public Veicolo(String marca, int anno) {  
        this.marca = marca;  
        this.anno = anno;  
    }  
    // Getter e Setter  
    public String getMarca() {  
        return marca;  
    }  
    public void setMarca(String marca) {  
        this.marca = marca;  
    }  
    public int getAnno() {  
        return anno;  
    }  
    public void setAnno(int anno) {  
        this.anno = anno;  
    }  
}
```

```
// Metodo per stampare i dettagli del veicolo
public void stampaDettagli() {
    System.out.println("Marca: " + marca + ", Anno: " + anno);
}

public class Auto extends Veicolo
{
    private String modello;
    // Costruttore
    public Auto(String marca, int anno, String modello)
    {
        super(marca, anno);
        this.modello = modello;
    }
    // Costruttore di copia
    public Auto(Auto altraAuto)
    {
        super(altraAuto.getMarca(), altraAuto.getAnno());
        //Chiamata al costruttore della classe base con i valori
        //ottenuti dai getter della classe base
        this.modello = altraAuto.getModello();
        // Utilizzo del getter della classe derivata per ottenere
        //il valore dell'attributo modello
    }
    // Getter e Setter
    public String getModello()
    {
        return modello;
    }
}
```

```
    public void setModello(String modello)
    {
        this.modello = modello;
    }

    // Metodo per stampare i dettagli dell'auto
    @Override
    public void stampaDettagli()
    {
        super.stampaDettagli();
        System.out.println("Modello: " + modello);
    }
}

// Classe principale
public class Main {
    public static void main(String[] args) {
        Auto auto1 = new Auto("Audi", 2025, "A5");
        auto1.stampaDettagli();
        // Utilizzo del costruttore di copia
        Auto auto2 = new Auto(auto1);
        auto2.stampaDettagli();
    }
}
```

Perché è importante l'incapsulamento (anche in caso di Ereditarietà)?

- **PROTEZIONE DEI DATI:** gli attributi privati possono essere modificati solo mediante metodi controllati. Si evitano modifiche non desiderate ai valori che gli attributi possono assumere (integrità dei dati).
- **CONTROLLO DELL'ACCESSO:** si possono definire regole precise su come i dati possono essere letti o modificati, rendendo il codice più robusto e sicuro.

Quando si parla di chi può accedere agli attributi di una classe, ci si riferisce al fatto che qualsiasi altro pezzo di codice può accedere e modificare gli attributi: ciò significa che qualsiasi classe o metodo all'interno dello stesso programma può cambiare i valori degli attributi senza restrizioni.

Si immagini, per esempio, di lavorare su un progetto condiviso con altri programmatori. Se gli attributi sono pubblici, ogni sviluppatore può modificare questi attributi direttamente. Ciò potrebbe portare a situazioni in cui i dati vengono cambiati in modo non previsto o non controllato, causando bug o comportamenti inattesi.

EREDITARIETÀ E COSTRUTTORI

Dal momento che ciascuna sottoclasse contiene un'istanza della classe genitore, la seconda classe (ovvero la classe genitore) deve essere inizializzata prima che venga inizializzata la classe figlia.

Il compilatore chiama automaticamente il costruttore di default della classe genitore ogni volta che la classe figlia viene creata. Nello specifico, questa chiamata è inserita come prima istruzione nel costruttore della classe figlia. Se nella classe genitore è stato disabilitato il costruttore di default perché ne sono stati definiti altri (per esempio il costruttore parametrizzato/di copia/non parametrizzato), allora nella classe figlia il costruttore della classe genitore deve essere chiamato in modo esplicito.

Esempio 1:

```
public class Car
{
    private boolean isON;
    private String licensePlate;
    //Costruttore di default abilitato
}
public class SDCar extends Car
{
    boolean selfDriving;
    //Costruttore auto-generato dalla piattaforma di sviluppo
    public SDCar() {
        super();
        //Si riferisce al costruttore di default della classe Car
    }
}
```

Esempio 2

```
public class Car
{
    private boolean isON;
    private String licensePlate;
    public Car()
    {
        this.isON=false;
        this.licensePlate="EW 912DZ";
    }
}

public class SDCar extends Car
{
    private boolean selfDriving;

    public SDCar(boolean selfDriving) {
        super();
        this.selfDriving = selfDriving;
    }
}
```

```
public class Veicolo {
    private double velocita;
    private String colore;
    Veicolo(double velocita, String colore) {
        this.velocita = velocita;
        this.colore = colore;
    }
    public void accelera() {
        System.out.println("Il veicolo accelera.");
    }
}

public class Moto extends Veicolo {
    private String tipoMotore;
    Moto(double velocita, String colore, String tipoMotore) {
        super(velocita, colore);
        this.tipoMotore = tipoMotore;
    }
    @Override
    public void accelera() {
        super.accelera();
        System.out.println("La moto accelera rapidamente con motore " + tipoMotore);
    }
}
```



```
public class TestVeicolo {  
    public static void main(String[] args) {  
        Veicolo mioVeicolo = new Veicolo(100, "rosso");  
        System.out.println("Veicolo: velocità = " + mioVeicolo.velocita  
+ ", colore = " + mioVeicolo.colore);  
  
        mioVeicolo.accelera();  
  
        Moto miaMoto = new Moto(150, "nero", "4 tempi");  
        System.out.println("Moto: velocità = " + miaMoto.velocità + ",  
colore = " + miaMoto.colore + ", tipo motore = " + miaMoto.tipoMotore);  
  
        miaMoto.accelera();  
    }  
}
```

BUONA REGOLA DI PROGRAMMAZIONE

Per implementare correttamente l'Ereditarietà conviene definire un costruttore all'interno di tutte le classi, sia nella classe genitore sia nelle classi figlie.

Ricorda che...

this: riferimento/puntatore all'oggetto corrente.

super: riferimento alla classe genitore.

super(): chiamata al costruttore di default/non parametrizzato della classe genitore.

super(parametri): chiamata al costruttore parametrizzato della classe genitore.

Come vengono costruiti gli oggetti figli?

L'esecuzione dei costruttori procede secondo uno schema top down lungo la catena dell'ereditarietà.

In questo modo, quando un metodo della classe figlia viene eseguito (incluso il costruttore), la superclasse è già completamente inizializzata.

EREDITARIETÀ – CLASSI FINAL

Una classe `final` non può essere estesa mediante ereditarietà: non è possibile creare sue sottoclassi.

Esempio:

```
public final class Veicolo
{
    ...
    ...
}

public class Moto extends Veicolo
{
    /* errore di compilazione */
}
```

ERROR: cannot inherit from final Veicolo.

Perché implementare classi final?

- **SICUREZZA:** dichiarare una classe come `final` garantisce che nessun'altra classe possa estenderla, impedendo modifiche accidentali o intenzionali alla struttura e al comportamento della classe.
- **PRESTAZIONI:** le classi `final` possono permettere al compilatore di effettuare ottimizzazioni. Il compilatore sa che la classe non sarà mai derivata e, pertanto, potrà compiere ottimizzazioni come l'**INLINING DEI METODI**, riducendo il sovraccarico delle chiamate ai metodi e migliorando le prestazioni complessive del codice.

COS'È L'INLING DEI METODI?

L'*inlining dei metodi* è un'ottimizzazione del compilatore che sostituisce una chiamata ha un metodo con il corpo del metodo stesso.

Il compilatore copia il codice del metodo direttamente nel punto in cui il metodo viene chiamato, eliminando la necessità di una chiamata al metodo stesso.

Ciò può migliorare le prestazioni del programma, in particolare riducendo il tempo di esecuzione perché evita il costo di passare i parametri e gestire il ritorno dal metodo.

L'inlining è spesso usato per metodi “piccoli” (poche righe di codice nel corpo del metodo) e frequentemente chiamati.

METODI DI TIPO FINAL

Quando un metodo è dichiarato `final` significa che non può essere sovrascritto, ovvero nessuna classe può cambiarne l'implementazione, comprese le sottoclassi.

Esempio: si immagina di avere un metodo che calcola l'interesse su un prestito. Se questo metodo è definito `final`, tutte le volte che verrà invocato, calcolerà l'interesse nello stesso modo, senza eventuali modifiche o sovrascrizioni da parte di altre classi.

```
public class CalcolaTasse{
    public final double calcoloTasse(double importo){
        return importo*0.20;
    }
}

public class CalcolaTasseSpeciali extends CalcolaTasse{
    public double calcoloTasse(double importo){
        return importo*0.18; // tentativo di override
    }
}
```